

Assignment 5 — Catching Data Leakage Before It Catches You

Feature Engineering & MLOps · Data Leakage: Target Leakage & Preprocessing Leakage

Course	Feature Engineering & MLOps
Assignment No.	5
Topic	Data Leakage — Target Leakage and Preprocessing Leakage
Marks	____ / 100
Due Date	_____

1. Objective

Data leakage is like **giving a student the answer key before the exam** — the model scores beautifully in testing, then quietly fails the moment it meets real, unseen data. This assignment focuses on the two most common ways leakage sneaks into a real project: **target leakage** (a feature that's secretly a result of the label, not a predictor of it) and **preprocessing leakage** (fitting a transformation on data the model should never have seen yet).

By the end, you should be able to demonstrate BOTH kinds of leakage with your own numbers, and — more importantly — correctly fix both.

2. Learning Outcomes

- Identify a target-leaked feature using the "could this exist at prediction time?" test.
- Demonstrate, with real numbers, how a leaked feature produces an unrealistically high validation score.
- Explain concretely why a model trained with target leakage fails in production.
- Demonstrate preprocessing leakage by comparing a scaler fit on the full dataset vs. fit only on the training split.
- Correctly reorder a preprocessing + train/test split pipeline to eliminate both kinds of leakage.

3. Dataset

Use `data/raw/customer_churn_a5.csv` — 600 telecom customers.

Column	Description
tenure_months	How many months this customer has held their account (legitimate)
monthly_charges	Current monthly bill amount (legitimate)
contract_type	Month-to-month / One year / Two year (legitimate)
support_calls	Number of support calls placed so far (legitimate)
days_since_cancellation	Days since the account was cancelled — ONLY present if churn=1
final_bill_amount	Closing balance calculated at account closure — ONLY present if churn=1
churn	Target — did this customer cancel their service?

4. Tasks

4.1 — Target leakage: prove it, don't just assert it

1. Load the dataset. Report how many rows have a non-null `days_since_cancellation`, and confirm — with a single line of code, not by eyeballing — that this count exactly equals the number of `churn=1` rows.
2. Compute the correlation of `final_bill_amount` (after filling missing values with -1) against `churn`. Is it suspiciously high? Compare it to the correlation of a genuinely legitimate feature like `tenure_months`.
3. Train Model A using ONLY the legitimate features (`tenure_months`, `monthly_charges`, `contract_type`, `support_calls`). Report test accuracy and ROC-AUC.
4. Train Model B using the legitimate features PLUS `days_since_cancellation` and `final_bill_amount` (fill missing values with -1 first). Report test accuracy and ROC-AUC.
5. Report the gap between Model A and Model B. In 2-3 sentences, explain EXACTLY what would happen if Model B were deployed to score brand-new applicants who haven't decided anything yet.

4.2 — Preprocessing leakage: the wrong order vs. the right order

1. Using ONLY the legitimate numeric features (`tenure_months`, `monthly_charges`, `support_calls`), fit a `StandardScaler` on the ENTIRE dataset, THEN split into train/test. Report the scaler's learned `mean_` and `scale_`.
2. Now do it correctly: split into train/test FIRST, then fit a NEW `StandardScaler` on the training split ONLY. Report its `mean_` and `scale_`.
3. Report the numeric difference between the two scalers' means. Even if the gap looks small, explain in 2-3 sentences WHY the principle matters regardless of how large the gap happens to be on this particular dataset.
4. Rebuild your Task 4.2 preprocessing correctly using a single `sklearn.Pipeline` (imputer + scaler + model in one object), fit only on the training split, and confirm it produces the exact same scaler statistics as your manual "correct" version in Step 2.

4.3 — The fix

Using only the legitimate features and a properly ordered pipeline (split first, fit preprocessing on train only), retrain a final model. Report its test accuracy and ROC-AUC, and confirm it matches Task 4.1's Model A — this is the honest, deployable version of your model.

5. Submission Instructions

1. In your existing course GitHub repository, use (or create) the top-level `Assignments` folder.
2. Add your completed notebook as `Assignment5_DataLeakage.ipynb`. Run every cell top-to-bottom with no errors, and keep outputs visible before committing.
3. Structure the notebook with clear markdown headers matching Sections 4.1-4.3 above.
4. Commit and push your changes:

```
git add Assignments/Assignment5_DataLeakage.ipynb
git commit -m "Assignment 5: Target and preprocessing leakage"
git push
```

Finally, share your GitHub repository link with the instructor by the due date above. Grading is done directly from the `Assignments` folder — make sure `Assignments 1` through `5` are all there and pushed before the deadline.

IMPORTANT: Only the notebook committed to your `Assignments` folder on GitHub will be evaluated. Local-only or emailed notebooks will not be graded.

6. Grading Rubric

Criteria	Weight
Target leakage correctly demonstrated with correlation + accuracy/AUC gap (4.1)	25%
Correct explanation of WHY the leaked model fails in production	15%
Preprocessing leakage correctly demonstrated (wrong vs. correct scaler) (4.2)	20%
Pipeline correctly rebuilt and matches the manual "correct" version	15%
Final fixed model (4.3) correctly matches the honest baseline from 4.1	15%
Code quality: clear structure, markdown headers, readable variable names	10%

7. Bonus (Optional, +10% extra credit)

Using scikit-learn's `Pipeline` and `cross_val_score`, show that even CROSS-VALIDATION can leak if preprocessing is fit before the CV split. Fit a `StandardScaler` on the whole dataset, then run 5-fold cross-validation on the scaled data — compare the resulting scores against properly wrapping the scaler INSIDE the `Pipeline` passed to `cross_val_score`. Explain any difference you find (or explain why the difference might be small on this particular dataset, even though the risk is still real in general).